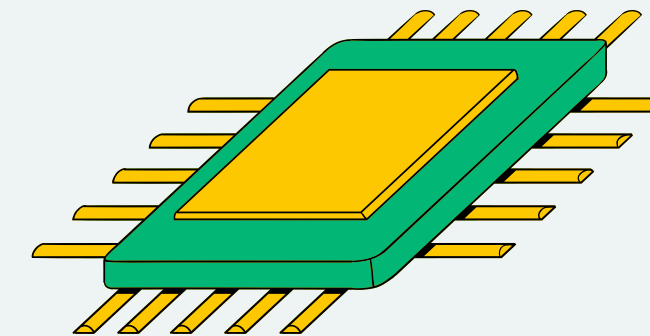
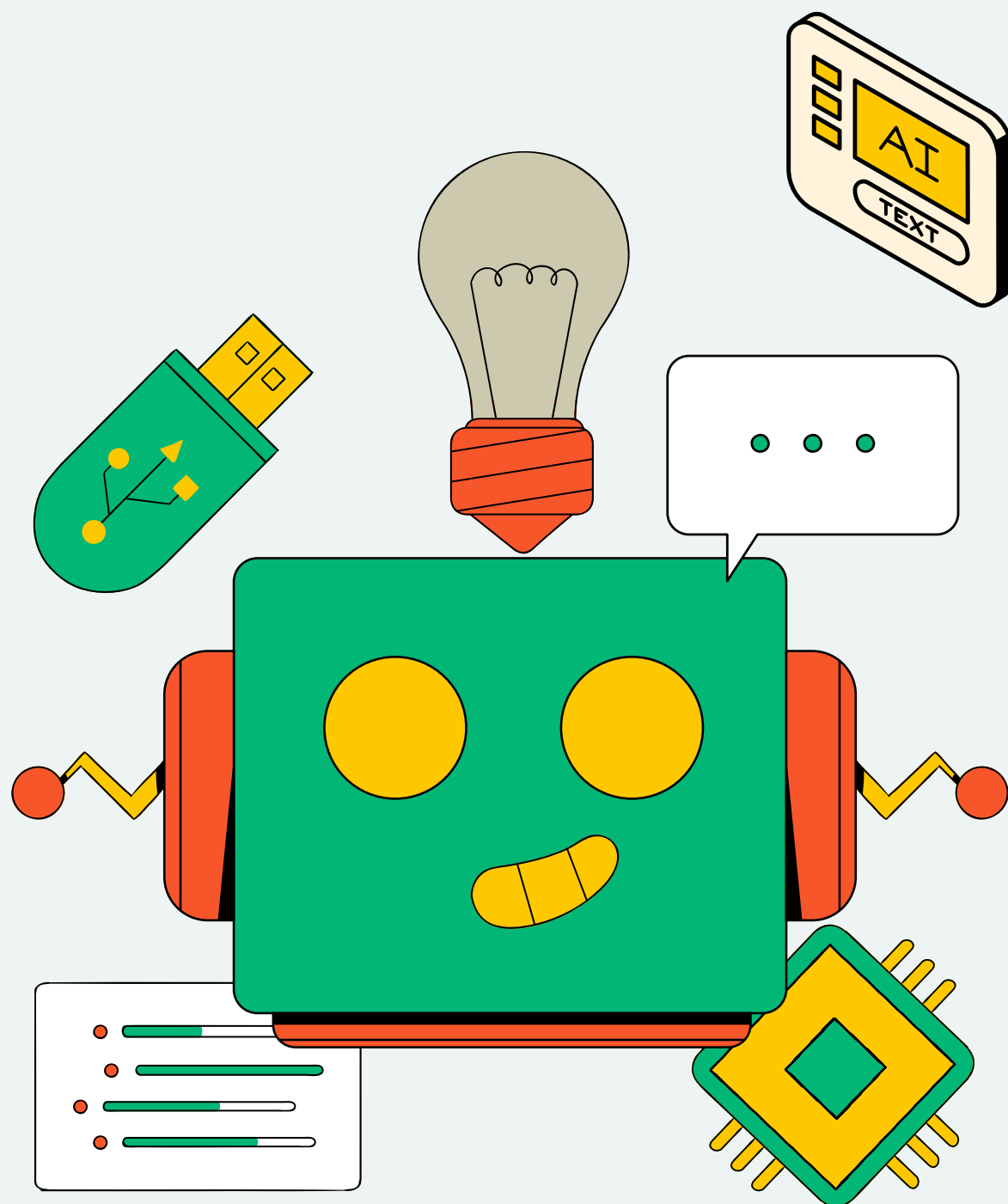


THYNK UNLIMITED
WE LEARN FOR THE FUTURE



KLASIFIKASI SENTIMEN, TOXICITY, DAN EMOSI PADA TEKS FORUM MENGGUNAKAN MODEL MACHINE LEARNING BERBASIS BAHASA INDONESIA



PRESENTED BY:

MATTHEW YUSLIH

SEBASTIAN OBERT

JANSEN CAESAR

PIERRE RAMBE

MANUEL MANATAP



PRESENTATION OUTLINE

- Latar Belakang
- Dataset
- EDA
- Preprocessing
- Model Train
- Model Evaluation
- Model Validation



LATAR BELAKANG

- Di era media sosial, netizen Indonesia aktif mengekspresikan pendapat terhadap isu publik dari politik, kebijakan, hingga fenomena sosial.
- Berbagai penelitian analisis sentimen (sentiment analysis) menunjukkan bahwa bagian dari komentar tersebut bersifat negatif, dan proporsi negatif ini tergantung topik & konteks.



Berdasarkan data dari studi kasus Tagar Indonesia Gelap, dari 64.816 komentar pada tahun 2025 didapatkan sekitar 81% komentar negatif, 8% positif, dan 13 % netral.



DATASET

Dataset Emotion

	label	tweet
0	anger	Soal jln Jatibaru,polisi tdk bs GERTAK gubernu...
1	anger	Sesama cewe lho (kayaknya), harusnya bisa lebi...
2	happy	Kepingin gudeg mbarek Bu hj. Amad Foto dari go...
3	anger	Jln Jatibaru,bagian dari wilayah Tn Abang.Peng...
4	happy	Sharing pengalaman aja, kemarin jam 18.00 bata...

Dataset Info:
- Jumlah Baris: 4401
- Jumlah Kolom: 2

Dataset Sentiment

	kalimat	sentiment
0	Kalimat-kalimat tersebut adalah sebagai berikut	0
1	Manusia membutuhkan makanan dan air supaya men...	1
2	Tanaman hijau menggunakan air untuk membuat ma...	0
3	Tanaman yang tidak mendapat air akan layu dan ...	1
4	Keberhasilan belajar murid tidak hanya bergantung...	1

Dataset Info:
- Jumlah Baris: 12609
- Jumlah Kolom: 2

Dataset Toxicity

	Tweet	HS	Abusive	HS_Individual	HS_Group	HS_Religion	HS_Race	HS_Physical	HS_Gender	HS_Other	HS_Weak	HS_Moderate	HS_Strong
0	- disaat semua cowok berusaha melacak perhatia...	1	1	1	0	0	0	0	0	1	1	0	0
1	RT USER: USER siapa yang telat ngasih tau elu?...	0	1	0	0	0	0	0	0	0	0	0	0
2	41. Kadang aku berfikir, kenapa aku tetap perc...	0	0	0	0	0	0	0	0	0	0	0	0
3	USER USER AKU ITU AKU\n\nKU TAU MATAMU SIPIT T...	0	0	0	0	0	0	0	0	0	0	0	0
4	USER USER Kaum cebong kapir udah keliatan dong...	1	1	0	1	1	0	0	0	0	0	1	0

Dataset Info:
- Jumlah Baris: 13169
- Jumlah Kolom: 13



EDA (EXPLORATORY DATA ANALYSIS)

1. Melakukan pengecekan Nan Values dan duplicate column pada masing masing dataset

```
--- DATASET: Emotion ---  
Ukuran Data: 4401 baris, 2 kolom  
Tidak ada missing values.  
Ada 11 data duplikat!  
Kolom tersedia: ['label', 'tweet']  
-----  
  
--- DATASET: Sentiment ---  
Ukuran Data: 12609 baris, 2 kolom  
Tidak ada missing values.  
Ada 1516 data duplikat!  
Kolom tersedia: ['kalimat', 'sentiment']  
-----  
  
--- DATASET: Toxicity ---  
Ukuran Data: 13169 baris, 13 kolom  
Tidak ada missing values.  
Ada 125 data duplikat!  
Kolom tersedia: ['Tweet', 'HS', 'Abusive'  
Moderate', 'HS_Strong']  
-----
```

Setelah dilakukan pengecekan, tidak ditemukan nilai NaN pada dataset yang kami gunakan. Namun, kami menemukan adanya duplicate column yang perlu dibersihkan agar proses training model tidak terganggu

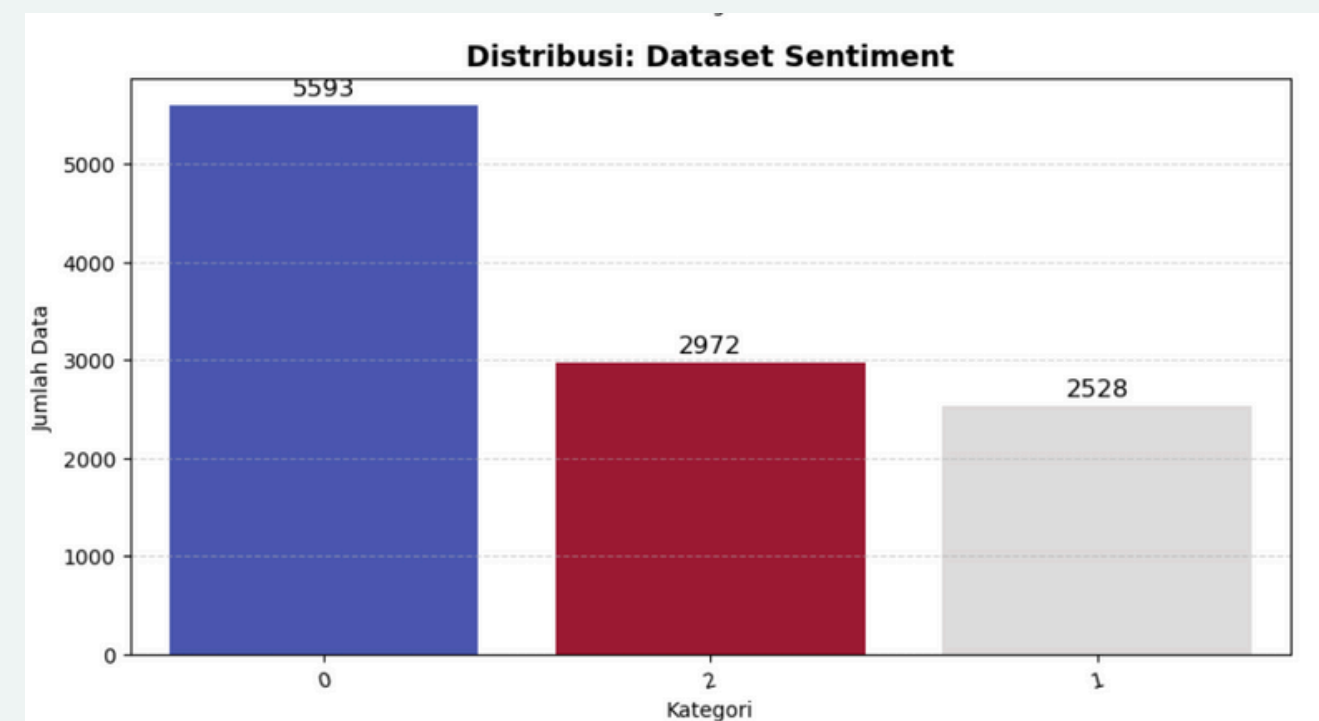
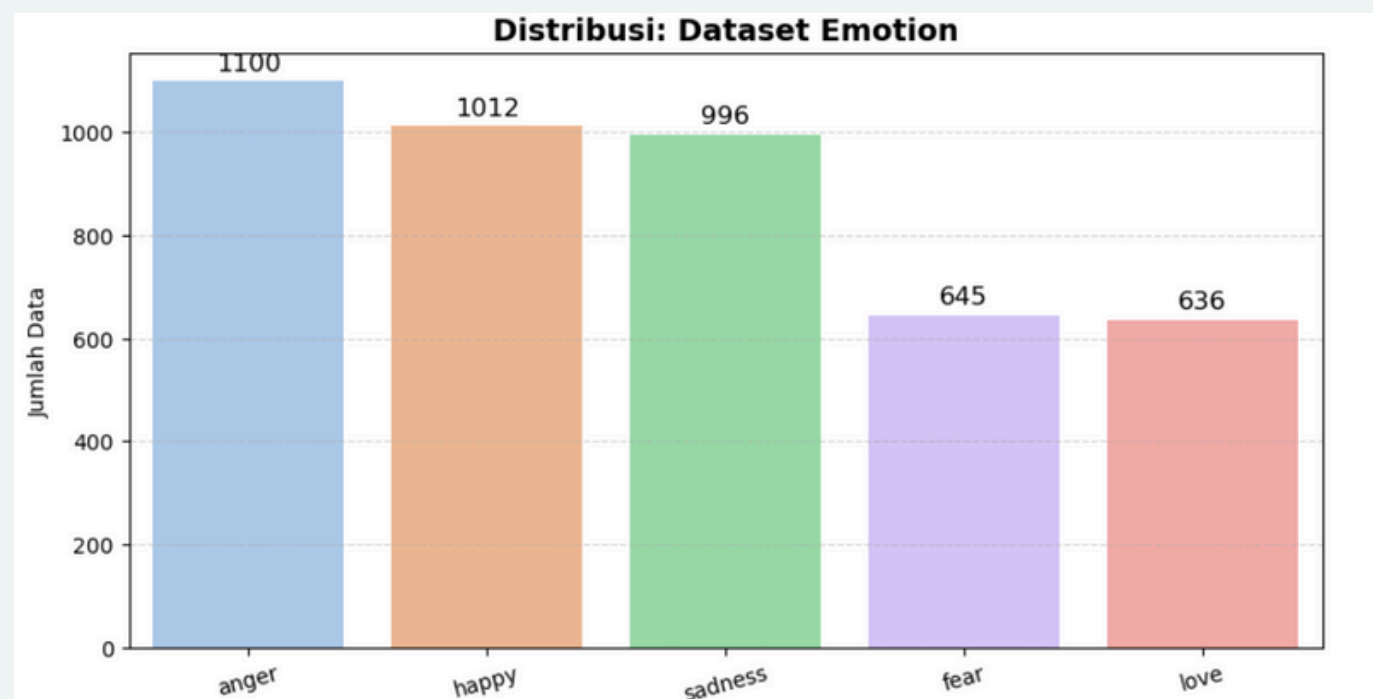
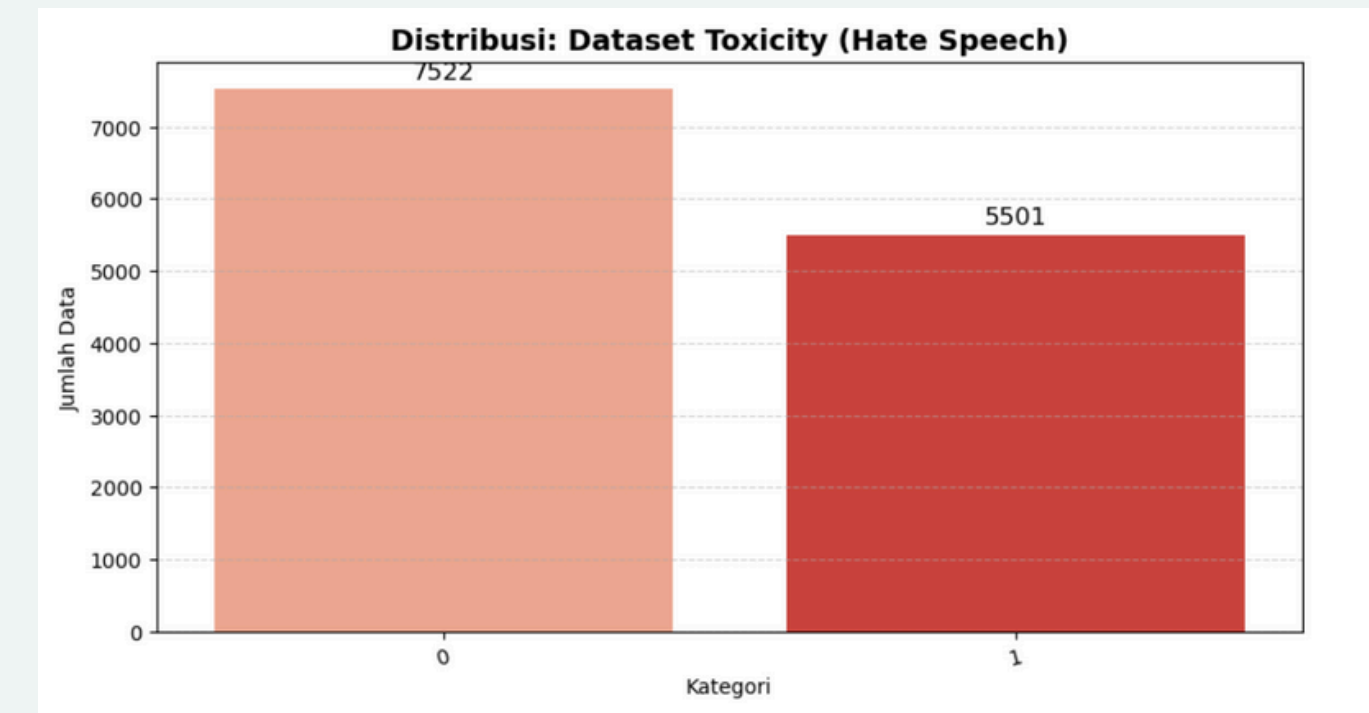
```
Emotion: Duplikat dibuang. Sisa 4389 data.  
Sentiment: Duplikat dibuang. Sisa 11093 data.  
Toxicity: Duplikat dibuang. Sisa 13023 data.
```



EDA (EXPLORATORY DATA ANALYSIS)

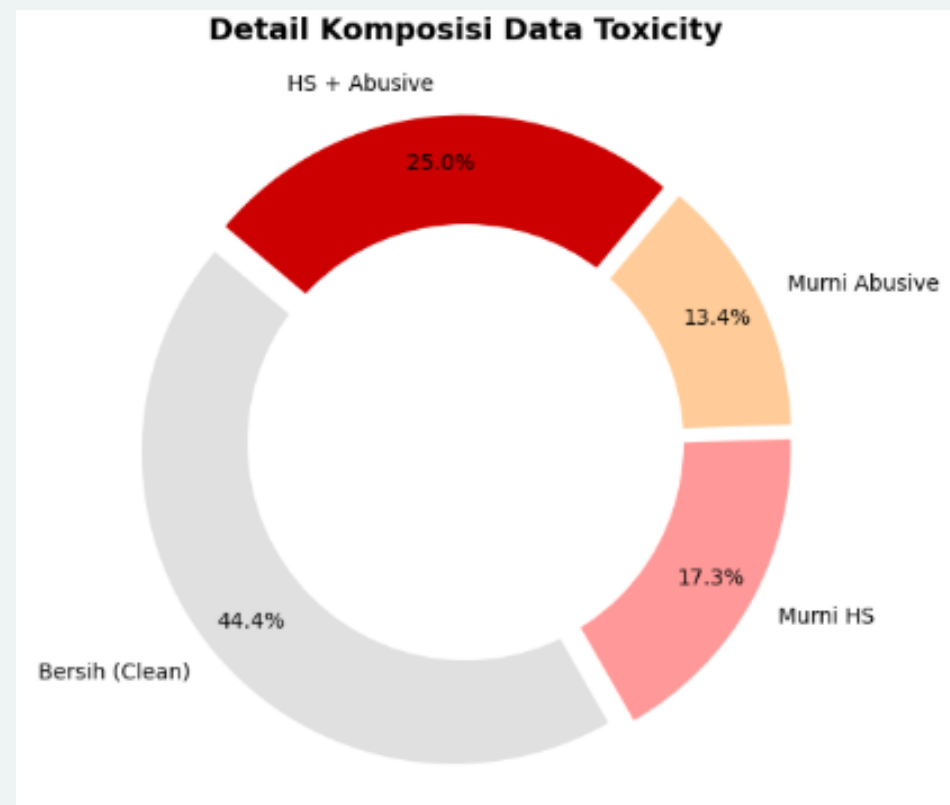
2. Visualisasi (EDA Chart)

Secara keseluruhan, visualisasi ini memperlihatkan karakteristik data yang beragam, yaitu Dataset Emosi cenderung tidak seimbang dengan dominasi kelas Anger, Happy, dan Sadness dibandingkan Fear dan Love, lalu Dataset Sentimen sangat condong ke arah Negatif (Label 0) yang wajar mencerminkan natur keluhan di media sosial, dan terakhir Dataset Toxicity sebagai target utama justru memiliki distribusi yang paling sehat dan cukup seimbang antara label Aman dan Toxic, sehingga kondisinya sangat ideal untuk melatih model agar objektif dalam mendeteksi ujaran kebencian.



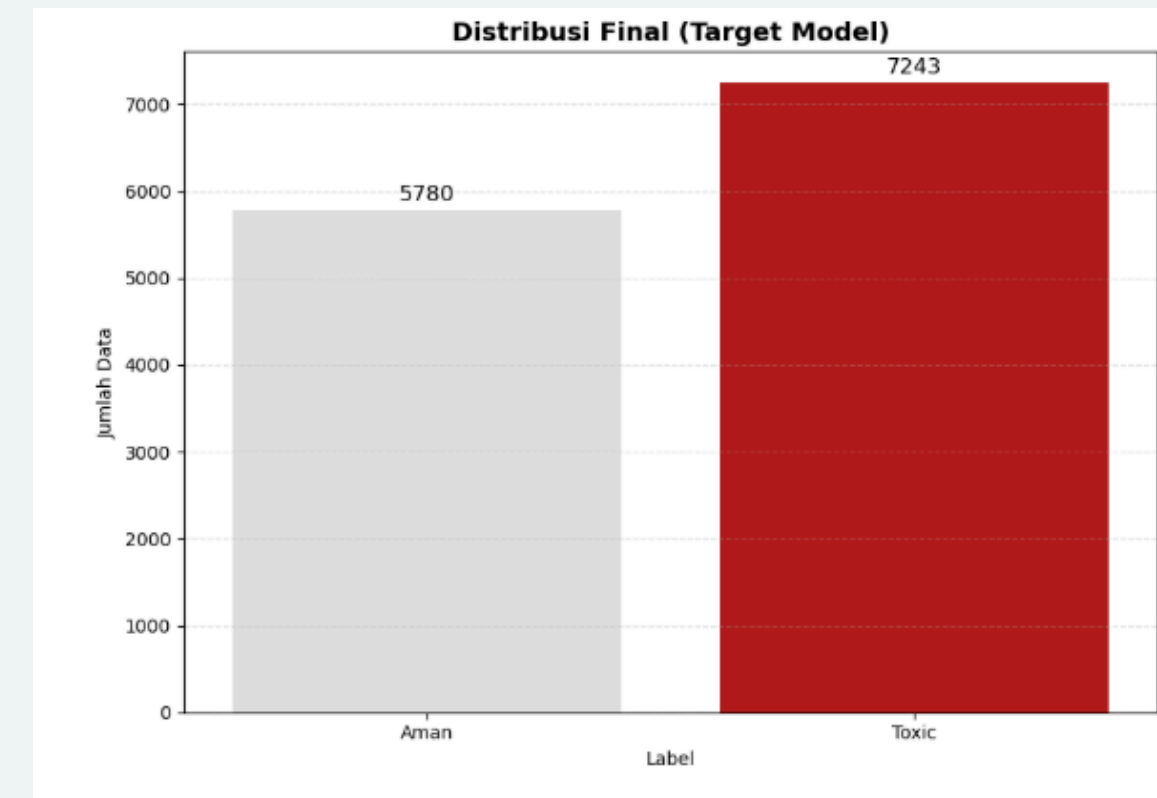
EDA (EXPLORATORY DATA ANALYSIS)

3. Analisis Distribusi Label Toxicity dan Penggabungan Label Toxicity



Visualisasi Donut Chart ini membedah kompleksitas data mentah, di mana komentar negatif ternyata tidak hanya satu melainkan terpecah menjadi tiga segmen, yaitu Hate Speech, Abusive, dan keduanya. Hal ini mengungkap temuan krusial bahwa terdapat porsi signifikan data Murni Abusive sebesar 13.4% yang berpotensi lolos deteksi jika kita hanya fokus pada ujaran kebencian, sehingga visualisasi ini menjadi landasan kuat mengapa penyederhanaan atau penggabungan label mutlak diperlukan demi cakupan deteksi yang lebih luas.

Grafik batang ini menampilkan hasil akhir transformasi data setelah seluruh varian negatif digabung menjadi satu label target tunggal yaitu Toxic. Terlihat bahwa rasio antara kelas Aman sebesar 5780 data dan Toxic sebesar 7243 data kini berada dalam proporsi yang cukup seimbang dan sehat. Kondisi distribusi ini sangat ideal untuk training model karena meminimalisir risiko model menjadi bias atau cenderung memprediksi kelas mayoritas saja.



PREPROCESSING

1. Data Cleaning

```
def bersihkan_teks(text):  
    # Case Folding  
    text = str(text).lower()  
  
    # Ganti Newline (\n) dengan spasi  
    text = text.replace('\n', ' ').replace('\n', ' ')  
  
    # Hapus Mention & URL  
    text = re.sub(r'@\w+', ' ', text)  
    text = re.sub(r'http\S+|www\S+|https\S+', ' ', text, flags=re.MULTILINE)  
  
    # Hapus kata "USER"  
    text = re.sub(r'\buser\b', ' ', text)  
  
    # Ganti Tanda Baca dengan Spasi  
    text = re.sub(r'[^w\s]', ' ', text)  
  
    # Hapus Angka  
    text = re.sub(r'\d+', ' ', text)  
  
    # Stopword Removal  
    kata_kata = text.split()  
    kata_bersih = [kata for kata in kata_kata if kata not in stopwords_set]  
  
    # Gabung Lagi  
    return " ".join(kata_bersih)
```

Tahap Data Cleaning ini bertujuan untuk menstandarisasi data teks mentah agar bebas dari noise yang dapat mengganggu akurasi model. Proses ini meliputi serangkaian teknik filtrasi, dimulai dari case folding untuk menyeragamkan huruf, penghapusan karakter non-alfanumerik (seperti tanda baca, angka, dan emoji), hingga eliminasi elemen tidak relevan seperti mention (@user), hashtag, dan tautan URL. Langkah ini krusial untuk memastikan bahwa algoritma hanya mempelajari inti kata yang bermakna, sehingga proses ekstraksi fitur selanjutnya menjadi lebih optimal dan bebas bias.



PREPROCESSING

2.Data Splitting

```
# 1. Split Emotion
X_train_emo, X_test_emo, y_train_emo, y_test_emo = bagi_dataset(df_emotion, 'text_clean', 'label', 'Emotion')

# 2. Split Sentiment
X_train_sen, X_test_sen, y_train_sen, y_test_sen = bagi_dataset(df_sentiment, 'text_clean', 'label', 'Sentiment')

# 3. Split Toxicity
X_train_tox, X_test_tox, y_train_tox, y_test_tox = bagi_dataset(df_toxicity, 'text_clean', 'final_label', 'Toxicity')
```

Proses Data Splitting dilakukan dengan membagi dataset menjadi dua bagian strategis menggunakan rasio 80:20, di mana 80% data dialokasikan untuk melatih model (training set) dan 20% sisanya disisihkan sebagai data uji (testing set). Pemisahan ini sangat krusial untuk mencegah kebocoran data sekaligus menjamin objektivitas evaluasi, sehingga performa model yang diukur nantinya benar-benar mencerminkan kemampuannya dalam memprediksi data baru yang belum pernah dilihat sebelumnya.



PREPROCESSING

3.Feature Extraction

```
# 1. TF-IDF Emotion
X_train_emo_tfidf, X_test_emo_tfidf, vect_emo = vektorisasi_tfidf(X_train_emo, X_test_emo, "Emotion")

# 2. TF-IDF Sentiment
X_train_sen_tfidf, X_test_sen_tfidf, vect_sen = vektorisasi_tfidf(X_train_sen, X_test_sen, "Sentiment")

# 3. TF-IDF Toxicity
X_train_tox_tfidf, X_test_tox_tfidf, vect_tox = vektorisasi_tfidf(X_train_tox, X_test_tox, "Toxicity")
```

Tahap Feature Extraction bertugas mengubah data teks mentah menjadi representasi numerik (matriks) yang dapat diproses oleh mesin menggunakan metode TF-IDF (Term Frequency-Inverse Document Frequency). Metode ini dipilih karena kemampuannya memberikan bobot lebih tinggi pada kata-kata spesifik yang menjadi ciri khas unik setiap kategori, sekaligus meredam dominasi kata-kata umum yang tidak bermakna, sehingga membantu saat training model membedakan pola antar kelas dengan jauh lebih akurat.



TRAINING MODEL

1. Training SVM Model (Sebelum Tuning)

```
def train_svm(X_train, y_train, X_test, y_test, topic):  
    print(f" [SVM] Sedang melatih dataset: {topic}...")  
  
    for _ in tqdm(range(100), desc=f"Training {topic}", ncols=80):  
        time.sleep(0.02)  
  
    svm = SVC(kernel='linear', random_state=42)  
    svm.fit(X_train, y_train)  
  
    acc = accuracy_score(y_test, svm.predict(X_test))  
  
    models_svm[topic] = svm  
    skor_akurasi[topic]['SVM'] = acc  
    print(f"Selesai! Akurasi: {acc*100:.2f}%")
```

Training model dimulai dengan algoritma Support Vector Machine (SVM) menggunakan konfigurasi kernel Linear. Algoritma ini dipilih sebagai model utama karena keunggulannya yang terbukti efektif dalam menangani ruang fitur berdimensi tinggi yang dihasilkan oleh TF-IDF. Pada fase ini, model dilatih menggunakan parameter standar (default) untuk menetapkan garis dasar (baseline) performa awal pada ketiga dataset sebelum dilakukan modifikasi apa pun.



TRAINING MODEL

2. Training Naive Bayes Model

```
def train_nb(X_train, y_train, X_test, y_test, topic):  
    print(f"[NB] Sedang melatih dataset: {topic}...")  
  
    # Simulasi Loading bar  
    for _ in tqdm(range(100), desc=f"Training {topic}", ncols=80):  
        time.sleep(0.01)  
  
    nb = MultinomialNB()  
    nb.fit(X_train, y_train)  
  
    acc = accuracy_score(y_test, nb.predict(X_test))  
  
    models_nb[topic] = nb  
    skor_akurasi[topic]['NB'] = acc  
  
    print(f"Selesai! Akurasi: {acc*100:.2f}%\n")
```

Training model juga dilakukan terhadap algoritma Naive Bayes yang berfungsi sebagai model pembandingan (benchmark). Algoritma ini sengaja kami pilih karena karakteristiknya yang ringan dan cepat, sehingga menjadi standar industri untuk mengukur efektivitas model yang lebih kompleks. Tujuannya adalah untuk membuktikan apakah penggunaan SVM benar-benar memberikan peningkatan akurasi yang signifikan dibandingkan metode probabilitas sederhana



TRAINING MODEL

3. Training SVM dengan Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV
from tqdm import tqdm
import time

print("Hyperparameter Tuning...\n")

all_grids = {}

def cari_setingan_terbaik(X_train, y_train, topic):
    print(f"\n{'='*40}")
    print(f"Sedang mengulik dataset: {topic}...\n")

    # Loading bar simulasi sebelum GridSearch
    for _ in tqdm(range(80), desc=f"Menelusuri Param {topic}", ncols=80):
        time.sleep(0.01)

    param_grid = {
        'C': [1, 10],
        'kernel': ['linear', 'rbf'],
        'gamma': ['scale']
    }

    print("\nMelakukan GridSearchCV (agak lama)...")
    grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=1, cv=3)
    grid.fit(X_train, y_train)

    print(f"\nSetingan Terbaik untuk {topic}: {grid.best_params_}")
    print(f"Skor Validasi: {grid.best_score_*100:.2f}%")

    # Update model terbaik
    models_svm[topic] = grid.best_estimator_

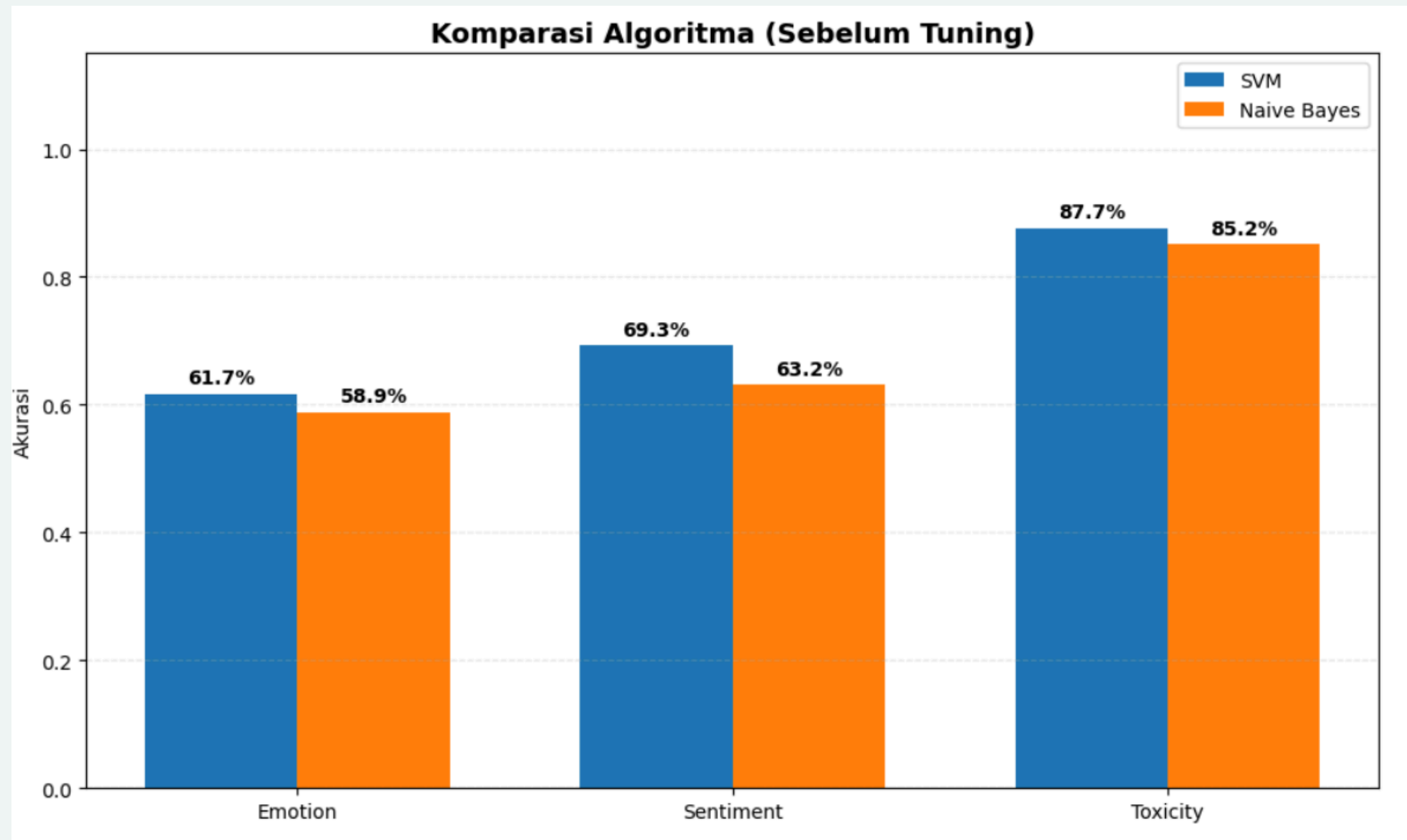
    return grid
```

Setelah SVM terbukti lebih unggul, dilakukan tahap Hyperparameter Tuning menggunakan teknik Grid Search untuk memaksimalkan performa dari model tersebut. Hasil dari tahap ini adalah sebuah model final yang tidak hanya memiliki akurasi optimal, tetapi juga lebih robust (tahan banting) dan stabil saat memprediksi data baru.



MODEL EVALUATION

1. Diagram perbandingan model SVM dan model NB

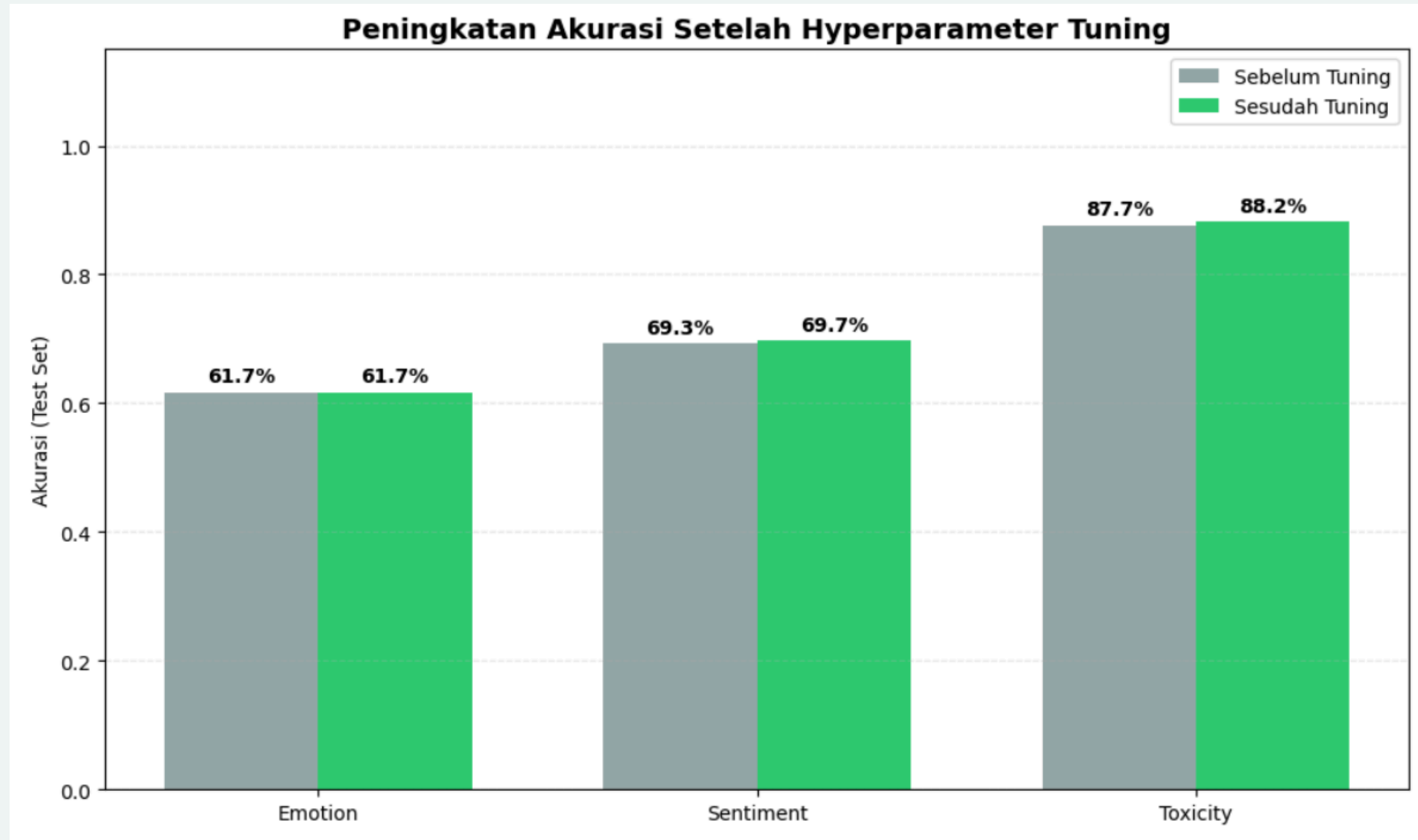


Berdasarkan hasil komparasi awal sebelum proses optimasi (pre-tuning), algoritma SVM terbukti secara konsisten mengungguli benchmark Naive Bayes di seluruh kategori pengujian (Emosi, Sentimen, dan Toksisitas). Grafik menunjukkan bahwa SVM mampu memberikan akurasi yang lebih tinggi secara merata, menegaskan bahwa model ini memiliki kemampuan generalisasi awal yang lebih baik dalam menangkap pola data dibandingkan metode probabilitas standar



MODEL EVALUATION

2. Diagram perbandingan Model SVM sebelum dan sesudah tuning



Grafik ini memvisualisasikan dampak akhir dari proses Hyperparameter Tuning. Setelah dilakukan optimasi, terlihat adanya peningkatan akurasi secara marginal pada kategori Sentiment (naik menjadi 69.7%) dan Toxicity (naik menjadi 88.2%). Sementara itu, kategori Emotion menunjukkan stabilitas performa yang konstan. Hasil ini mengonfirmasi bahwa proses tuning berhasil 'memeras' potensi maksimal algoritma untuk mencapai titik akurasi tertinggi yang bisa didapatkan dari dataset



MODEL EVALUATION

3. Angka detail model terbaik (SVM)

[EMOTION DATASET]				
	precision	recall	f1-score	support
anger	0.62	0.76	0.68	220
fear	0.78	0.60	0.68	129
happy	0.57	0.51	0.54	203
love	0.77	0.70	0.73	127
sadness	0.50	0.53	0.52	199
accuracy			0.62	878
macro avg	0.65	0.62	0.63	878
weighted avg	0.63	0.62	0.62	878
[SENTIMENT DATASET]				
	precision	recall	f1-score	support
Negatif	0.72	0.87	0.78	1117
Positif	0.67	0.49	0.57	505
Netral	0.66	0.55	0.60	594
accuracy			0.70	2216
macro avg	0.68	0.64	0.65	2216
weighted avg	0.69	0.70	0.69	2216
[TOXICITY DATASET]				
	precision	recall	f1-score	support
Aman	0.87	0.87	0.87	1155
Toxic	0.89	0.89	0.89	1449
accuracy			0.88	2604
macro avg	0.88	0.88	0.88	2604
weighted avg	0.88	0.88	0.88	2604

Untuk memvalidasi keandalan model secara objektif, kami melakukan prediksi pada Test Set (data yang belum pernah dilihat model) dan mengevaluasinya menggunakan `classification_report`. Kode ini membedah performa tidak hanya dari akurasi global, tetapi juga melalui metrik Precision, Recall, dan F1-Score. Hasil breakdown ini mengonfirmasi bahwa model Toxicity memiliki performa paling robust dengan akurasi 88%, sementara model Emotion menghadapi tantangan terbesar karena variasi kelas linguistik yang kompleks.

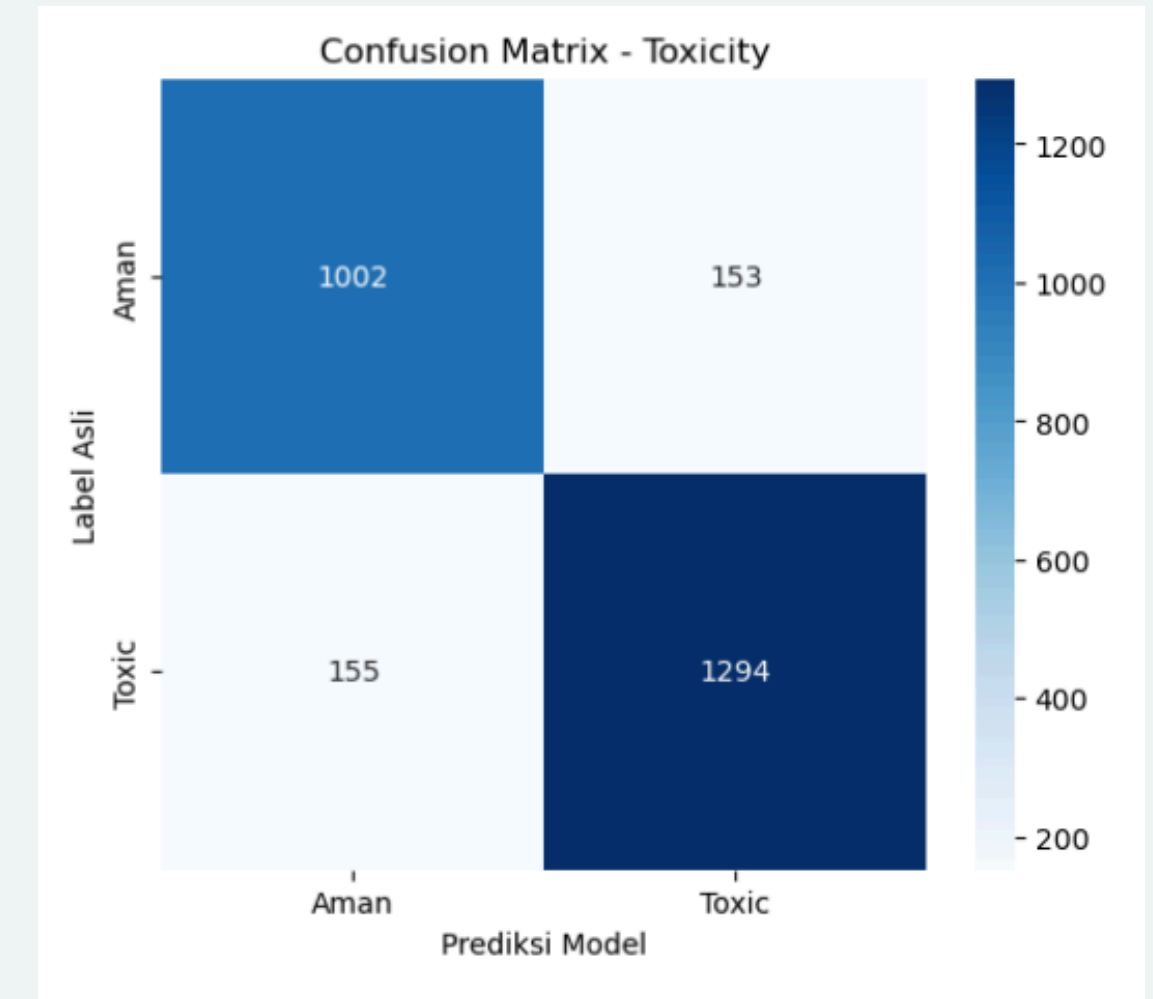
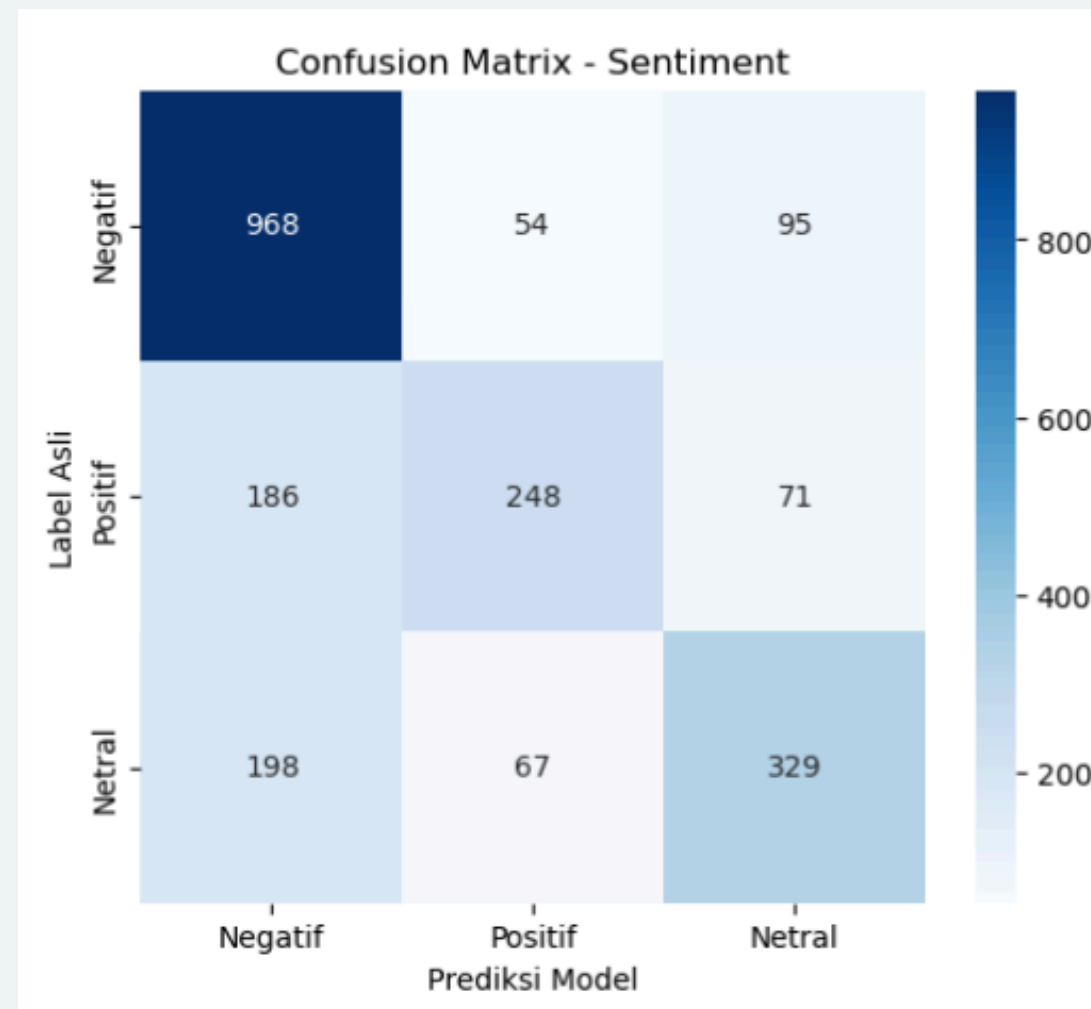
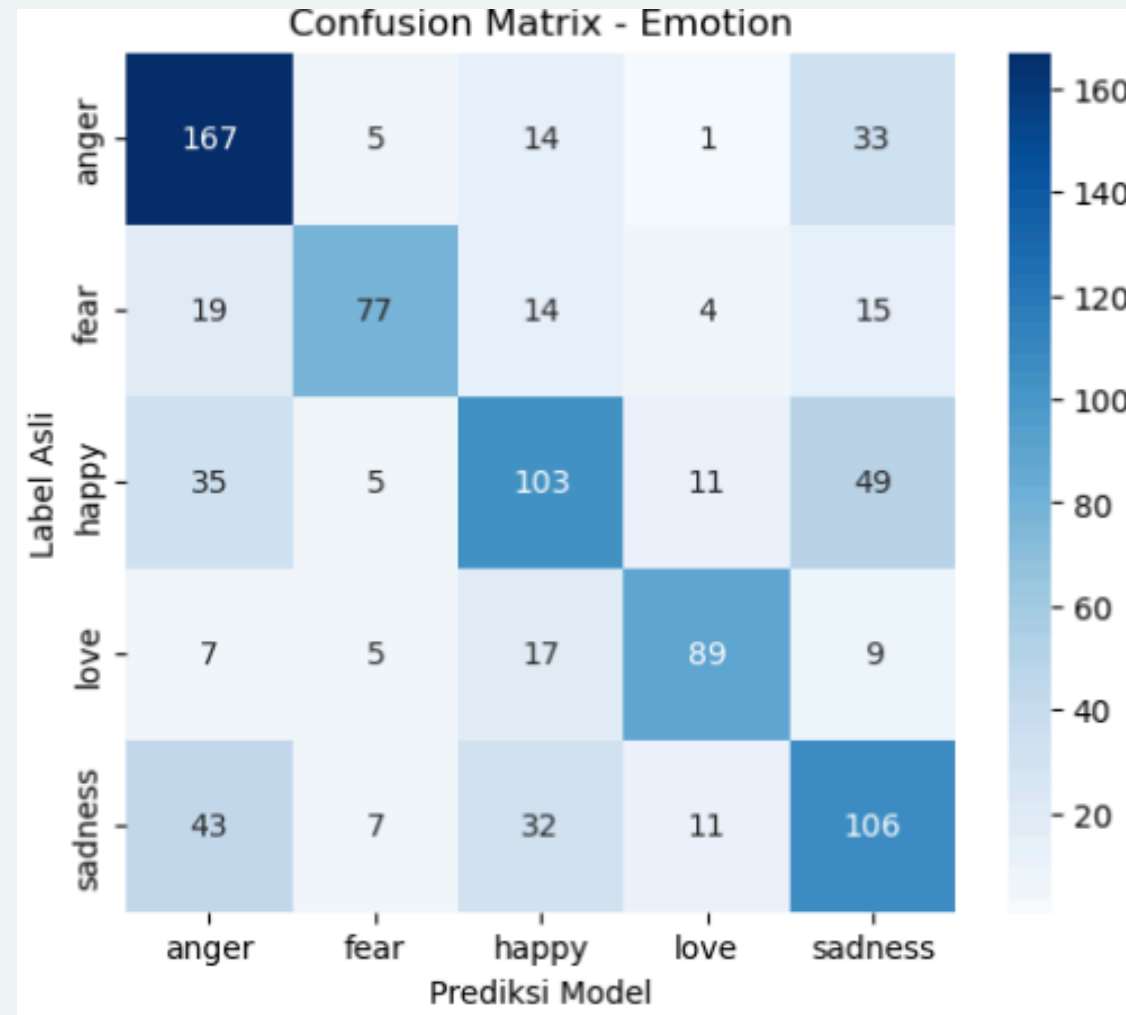


MODEL EVALUATION

4. Confussion Matrix



Confussion Matrix



kami memetakan performa model menggunakan Confusion Matrix untuk melihat di mana letak kesalahan prediksi secara spesifik. Warna biru pekat pada diagonal utama di ketiga grafik menunjukkan bahwa model secara konsisten memprediksi kelas yang benar. Khusus pada model Toxicity (kanan), terlihat pemisahan yang sangat tegas antara kelas 'Aman' dan 'Toxic' dengan tingkat kesalahan (false positive/negative) yang sangat minim, mengonfirmasi akurasi tinggi yang kita lihat sebelumnya.

Model Validation

```
def prediksi_final(kalimat_input):
    print(f"Input: '{kalimat_input}'")

    kalimat_clean = bersihkan_teks(kalimat_input)

    input_emo = vect_emo.transform([kalimat_clean])
    pred_emo = models_svm['Emotion'].predict(input_emo)[0]

    input_sen = vect_sen.transform([kalimat_clean])
    pred_sen_val = models_svm['Sentiment'].predict(input_sen)[0]
    map_sen = {0: "Negatif", 1: "Positif", 2: "Netral"}
    label_sen = map_sen.get(pred_sen_val, "Unknown")

    input_tox = vect_tox.transform([kalimat_clean])
    pred_tox_val = models_svm['Toxicity'].predict(input_tox)[0]
    label_tox = "Toxic" if pred_tox_val == 1 else "Aman"

    print(f"Hasil Prediksi Machine Learning")
    print(f"    • Emosi      : {pred_emo.upper()}")
    print(f"    • Sentimen   : {label_sen}")
    print(f"    • Toksisitas : {label_tox}")
    print("-" * 40)

    prediksi_final("Jansen orang baik")
    prediksi_final("Seneng banget akhirnya bisa lulus skripsi tahun ini, makasih tuhan.")
    prediksi_final("Hujan terus dari pagi, jadi batal main bola deh.")
```

Input: 'Jansen orang baik'

Hasil Prediksi Machine Learning

- Emosi : ANGER
- Sentimen : Positif
- Toksisitas : Aman

Input: 'Seneng banget akhirnya bisa lulus skripsi tahun ini, makasih tuhan.'

Hasil Prediksi Machine Learning

- Emosi : HAPPY
- Sentimen : Positif
- Toksisitas : Aman

Input: 'Hujan terus dari pagi, jadi batal main bola deh.'

Hasil Prediksi Machine Learning

- Emosi : HAPPY
 - Sentimen : Negatif
 - Toksisitas : Aman
-

kami menguji sistem dalam skenario penggunaan nyata (real-world simulation). Kami membangun fungsi prediksi_final yang mengintegrasikan ketiga model SVM menjadi satu pipeline eksekusi. Fungsi ini menerima input teks mentah, memprosesnya, dan memberikan tiga label analisis sekaligus. Seperti terlihat pada hasil uji coba, model mampu menangkap konteks dengan tepat— contohnya kalimat tentang 'lulus skripsi' yang sukses diklasifikasikan sebagai Happy, Positif, dan Aman. Ini membuktikan bahwa sistem sudah siap untuk diimplementasikan ke dalam aplikasi.

TERIMA KASIH